

Backend Development

FastAPI routes, Pydantic models and database schema

Backend Development Documentation

Project: FitBuddy — AI-Powered Personal Fitness Coach

Overview

The FitBuddy backend is a **FastAPI** application (`backend/main.py`) that:

- 1. Serves the frontend static files and HTML page
- 2. Exposes RESTful API endpoints for the 3 AI scenarios
- 3. Integrates with **Google Gemini 2.0 Flash** for plan generation
- 4. Stores user data in a **SQLite** database

File Structure

```
backend/ ├── main.py # FastAPI application – all routes, logic, DB
          ├── requirements.txt # Python dependencies
          ├── fitbuddy.db # SQLite database (auto-created on first run)
          ├── __pycache__/ # Python bytecode cache
```

1. Technology Stack

Component	Library	Version
Web Framework	FastAPI	0.115.5
ASGI Server	Uvicorn	0.32.1

AI Integration	google-generativeai	0.8.3
Data Validation	Pydantic	2.10.3
File Upload	python-multipart	0.0.17
Async I/O	aiofiles	24.1.0
Database	SQLite3	(built-in)

2. Configuration

```
GEMINI_API_KEY = os.getenv("GEMINI_API_KEY", "fallback-key") genai.configure(api_key=GEMINI_API_KEY)
model = genai.GenerativeModel("gemini-2.0-flash")
```

Environment Variable: Set `GEMINI_API_KEY` before running the server.

```
env: GEMINI_API_KEY = "your-api-key-here"
```

3. Database Schema

Table: `users`

Column	Type	Description
id	INTEGER PK	Auto-increment user ID
name	TEXT	User's full name
age	INTEGER	User's age in years
weight	REAL	User's weight in kg
goal	TEXT	Fitness goal (weight_loss / muscle_gain / general_wellness)
intensity	TEXT	Workout intensity (low / medium / high)

created_at	TEXT	ISO format timestamp
------------	------	----------------------

Table: `workout_plans`

Column	Type	Description
id	INTEGER PK	Auto-increment plan ID
user_id	INTEGER FK	References users.id
plan_json	TEXT	Full plan as JSON string
feedback	TEXT	Feedback used to generate this plan (empty for initial)
created_at	TEXT	ISO format timestamp

Table: `nutrition_tips`

Column	Type	Description
id	INTEGER PK	Auto-increment tip ID
user_id	INTEGER FK	References users.id
goal	TEXT	Goal this tip addresses
tip	TEXT	Full tip data as JSON string
created_at	TEXT	ISO format timestamp

4. Pydantic Models

UserProfile

```
class UserProfile(BaseModel): name: str age: int weight: float goal: str # weight_loss | muscle_gain | general_wellness intensity: str # low | medium | high
```

FeedbackRequest

```
class FeedbackRequest(BaseModel): user_id: int feedback: str
```

NutritionRequest

```
class NutritionRequest(BaseModel): user_id: int goal: str
```

5. API Endpoints

POST /api/generate-plan — Scenario 1

Description: Generates a personalized 7-day workout plan for a new user.

Process:

1. Validate input data via Pydantic
2. Insert new user record into `users` table
3. Build personalized Gemini prompt with user profile
4. Parse AI JSON response
5. Store plan in `workout_plans` table
6. Return `user_id`, `plan_id`, and full plan data

Response Schema:

```
{ "success": true, "user_id": 1, "plan_id": 1, "plan": { "plan title": "...", "goal summary": "...",  
  "weekly_overview": "...", "days": [...], "weekly_tips": [...] } }
```

POST /api/update-plan — Scenario 2

Description: Updates an existing user's plan based on natural language feedback.

Process:

1. Validate `user_id` and feedback
2. Fetch existing user profile from database
3. Build updated prompt with original profile + feedback section
4. Parse AI JSON response
5. Store updated plan (with feedback logged)
6. Return updated plan

Response Schema:

```
{ "success": true, "plan_id": 2, "plan": {...}, "feedback_applied": "I want more cardio" }
```

POST /api/nutrition-tip — Scenario 3

Description: Generates personalized nutrition and recovery advice.

Process:

1. Look up user name from database (or use "User" if not found)
2. Build nutrition-focused Gemini prompt
3. Parse AI JSON response
4. Store tip in `nutrition_tips` table
5. Return nutrition data

Response Schema:

```
{ "success": true, "nutrition": { "tip_title": "...", "tip": "...", "foods_to_include": [...],  
  "foods_to_avoid": [...], "hydration_advice": "...", "recovery_advice": "..." } }
```

GET /api/user/{user_id}/history

Returns all workout plans and nutrition tips for a given user.

GET /api/health

Returns service health status:

```
{ "status": "ok", "service": "FitBuddy API", "version": "1.0.0" }
```

6. Middleware Configuration

```
app.add_middleware( CORSMiddleware, allow_origins=["*"], allow_credentials=True,  
  allow_methods=["*"], allow_headers=["*"], )
```

CORS is enabled for all origins to support development and deployment flexibility.

7. Static File Serving

```
app.mount("/static", StaticFiles(directory="../frontend/static"), name="static") @app.get("/") async  
def serve_frontend(): return FileResponse("../frontend/index.html")
```

The backend serves the frontend from `../frontend/` relative to the `backend/` directory.

8. Error Handling

Error Type	Handling
<code>json.JSONDecodeError</code>	Returns 500 with "AI response parsing error" detail
<code>HTTPException (404)</code>	User not found — returns 404
General <code>Exception</code>	Returns 500 with the exception message

All route handlers have try-except blocks returning appropriate HTTP status codes.

9. Running the Backend

```
# Navigate to backend directory cd backend # Install dependencies pip install -r requirements.txt #  
Set API key (PowerShell) $env:GEMINI_API_KEY = "your-gemini-api-key" # Start the server uvicorn  
main:app --reload --port 8000
```

Access the application at: <http://localhost:8000>

Access the API docs at: <http://localhost:8000/docs>